Project: Telco Customer Churn Dataset

Course: Machine Learning 1

Team Members: Brian Oommen, Toan, and Montserrat

Reflection Journal

Introduction - Brian

This project aimed to clean and prepare the Telco Customer Churn dataset for a future machine learning exercise. Before a machine learning model can use the data, we need to check for missing values, correct data types, and prepare numerical and categorical columns. I was able to also use Python and pandas to clean the data, while Matplotlib, and Seaborn helped us to create a visualization to understand the missing values, customer churn and numerical distributions. I also used a 70/30 split to separate the training data from the testing data so a future model can learn from one set and be evaluated on the other. Our preprocessing included filling missing values, encoding categories, scaling numerical features, creating features anew, and balancing the training data. I used sample rows, charts, and output checks to complete the data before and after preprocessing. I learned that preprocessing is important to clean and properly prepare the data to help machine learning models learn patterns more efficiently and give good predictions.

Description of the Dataset and Project - Brian

I worked with the Telco Customer Churn dataset, which has information about customers and whether they continued using a telecommunications company's services or left. Each row represents one customer, and each column shows a customer's characteristics, account details, or service. For example, I used columns such as tenure, which shows how long a customer has stayed with the company, and MonthlyCharges and TotalCharges, which show the monthly and total charges. I also worked with categorical columns such as InternetService, PaymentMethod, and Contract, which describe whether a customer left the company, making the target for a future classification model. The original dataset contains 7,043 rows, and I was able to work on cleaning and preparing the data.

I began by loading the dataset and displaying its first rows, shape, and data types so I could understand its formatting. I checked for missing values and blank spaces and was able to use visualizations to find columns that needed cleaning. I converted columns such as tenure, MonthlyCharges, and TotalCharges to numbers using the median and filled missing categorical values using the mode, based on the training data. I also created a tenure_group feature, encoded categorical columns, standardized the numerical features, and used random oversampling to even balance the training data. I then checked sample rows, compared distributions, and ran a validation check to make sure the data was ready for future training that will be done later.

Data Cleaning Decision and Theirs Reason – Brian, Toan and Montserrat

A. Removing CustomerID

I removed CustomerID from the model's input features because it is used to identify individual customers rather than describing their behavior. Although each customer has a unique ID, the number itself did not explain why a customer might leave the company. I wanted the model to bae able to focus on useful information such as contract type, monthly charges, and tenure. It also didn't make sense to keep Customer ID; it doesn't really help the model learn patterns in order to make useful predictions for customers. In the notebook, I removed these columns from the X_train and X_test, which have the input features for the future model. This allowed me to keep the focus on the customer infromation and more useful information to predict on.

B. Removing 1,000 Rows with Missing Internet Service

Our project instructions mention removing 1,000 rows with missing InternetService values, so I need to follow the instructor's direction if this removal is needed. If I remove these rows, the dataset will decrease from 7,403 to 6,403 rows, meaning I would lose 14.2% of the original dataset. However, I do get that removing these rows would help prevent me from making assumptions about customers whose internet service information is missing. Conversely, I would also lose information about these customers, which could reduce the amount of variety in the data available for training. In the end, removing the 1000 rows, I believe, was a good choice; as I said before, removing these rows would be better than assuming what could be filled in those data points.

C. Filling Missing Numerical Values

I converted the numerical columns such as tenure, MonthlyCharges, and TotalCharges to numerical values so Python could perform the calculations needed for the algorithm to use them. I learned that blank entries in TotalCharges can become missing values when I convert the column to numeric types. To handle missing numerical values, I used the available numbers. I chose the median because it is less affected by high or low values than the mean. I also calculated the median using the training data and applied the same value to the testing data. This helped me prevent information leakage because it does not use testing data to calculate the values needed to fill missing entries.

D. Filling Missing Categorical Values

I used mode imputation to fill missing values in categorical columns such as InternetService, PaymentMethod, and Contract. The mode is the category that seems to appear most in the training data, so I used it to replace missing categorical values. I chose to use this method because it allows the customer records to be kept instead of deleting them because one category is missing. I calculated the most common category from the training data and applied the same value to the testing data. However, I learned that mode imputation has its limitations because it can make the most common category appear more frequently than it originally did.

Even with the limitation, I found that mode imputation helps provide some consistent way to fill missing values, which helps keep the data available.

E. Encoding Categorical Data

I encoded categorical data because the machine-learning model needs numerical input instead of just text. In the notebook, I used one-hot encoding through pd.get_dummies() to convert the categories into separate columns containing 0s and 1s. For example, I converted categories in PaymentMethod,m InternetService, and Contract into different columns. I then aligned the testing columns with the training columns, so both datasets had the same features in the same order. If a category column existed in the training data but not in the testing data, I filled the missing testing column with zeros. I learned that one-hot encoding allows us to represent the categories without giving them numerical order, which could cause the model to be confused.

F. Encoding Churn

I encoded the Churn column because it is the target variable I want a future classification model to predict. I mapped NO to 0 and Yes to 1 so that the target would contain numerical labels instead of text. The value 0 represents a customer who stayed with the company, while 1 represents a customer who left. I separated these labels into y_train and y_test so the target could be included among the other input features. This separation allows the model to learn from

customer information and compare its prediction with actual churn labels. I learned that encoding the target is an important step because it gives the model a clear number to predict.

G. Scaling Numerical Features

I used StandardScaler to standardize tenure, MonthlyCharges, and TotalCharges. I was able to choose these values because these numerical data can have different ranges, and standardization puts them on a more reliable scale. StandardScaler is able to adjust the features using the mean and standard deviation from the training data, giving the scaled training values a mean near 0 and standard deviation near 1. I fitted the scaler on the training data only and then used the same fitted scaler to transform the testing data, which helped me prevent information leakage because the testing data would not influence the scaling calculations. I also learned that standardization and normalization are not the same, so I need to ask my instructor whether a separate normalization method is needed for the project.

H. Splitting and Balancing the Data

I used train_test_split() to divide the dataset into 70% training data and 30% testing data. I used stratification based on the Churn, so both sets would maintain a similar proportion of customers who stayed and customers who left. I also used random_state=42 to make the split reproducible, which could create the same split when using the same data and code. After preprocessing the training data, I used RandomOverSampler to balance the classes by adding repeated examples from the smaller churn class. I did this because fewer customers leave than

stay, and balancing can help the model learn from the smaller class. I balanced the training data and left the testing data unchanged, so I can evaluate the model fairly using the original class distribution.

Brian's Reflection

Developing this notebook was so interesting; it helped me understand how important it is to clean and prepare data before using it for machine learning. At first, I found it challenging to understand how different types of data needed to be processed. It took a while to learn how to handle missing values, convert numerical columns into the correct data types, and encode categorical data. It also required using logic to understand the data because, through logic, I could understand what to delete, when to use mode, median, mean, and when to use StandardScaler and one-hot encoding. One of the most important things I learned was why I should remove CustomerID from the model’s input features because it identifies customers but does not explain their behavior. I also learned how to use sample rows, visualizations, and output checks to some extent in order to see how data changed during preprocessing. These checks helped me find problems and understand why each cleaning step was so important. Overall, this experience helped me become more comfortable working with real-world data using Python.

I also learned how different preprocessing techniques affect the quality and usefulness of data. For example, I could use the median to fill in missing numerical values and the mode to fill missing categorical values instead of just deleting the customers' records. I used one-hot encoding to convert the categorical information into numerical columns and mapped Churn to 0

for No and 1 for Yes, so a future model can predict whether a customer leaves or not. Another important concept I learned was data leakage, which can happen when information from the testing data influences preprocessing or training data. I almost made a mistake where I scaled the data first before training it 70/30 split it could have let a leakage, so in order to prevent that I did the calculation for median and mode and fit the StandardScaler using the training data only before applying those values to the testing data. I also was able to learn 70/40 split and stratification are useful for keeping the training and testing sets separate while keeping similar churn proportions. These steps helped me connect what I learned in class to a practical machine learning project.

If I were to improve this project in the future, I would check every preprocessing step more carefully to make sure the final notebook follows the instructor's requirements. I would also pay attention to the missing InternetService values because removing these rows would help reduce the dataset, while filling them with the mode would preserve more customer records. However, even thinking about filling in with the mode would lead to an imbalance in the data, so the best option was to delete those blanks. I would also make sure that the numerical scaling meets project requirements and whether I need to separate normalization methods. Checking the output after each step would make sure the final dataset is processed correctly. I can also use these skills in future projects because missing values, categorical data, and data leakage are issues in many machine learning applications. Learning to use Python code for preprocessing also showed me how to make my work more consistent and be able to reuse it again.

Overall, this project helped improve my understanding of how to prepare a dataset for a future machine learning model. I learned about preprocessing, including more than just removing missing values, because every decision can help affect the information available to the model. I now understand why I need to choose the methods for numerical and categorical data and check the results before applying them. I also understand why I must keep the testing data separately to avoid any preprocessing decisions. The most valuable lesson I gained was to carefully preprocess and create a fairer and more reliable evaluation of a machine learning model. I would also apply that in future project assignments involving data analysis, feature engineering, and machine learning.

Toan's Reflection

During this lab, I learned a lot about working with a dataset in Google Colab and how to make the whole process easier. At first, I had trouble importing the dataset from Kaggle. I tried using the URL, but it did not work in my notebook, so I switched to uploading the file manually. The problem was that I had to upload it again every time I started a new session. After trying different methods, I decided to put the dataset in Google Drive and share access with my team members. Once I set everything up, the notebook could load the dataset directly from Drive whenever we ran the code. This experience taught me that sometimes solving a small technical problem can make the whole project much easier to manage.

I also learned more about how to organize code and clean dataset properly. While going through our notebook, I noticed duplicate records, missing values, and blank entries that needed to be checked. I helped rearrange the code so the steps would follow a clearer order and be easier to understand. I also realized that there can be different ways to handle missing values, and

choosing a method depends on the type of data and the situation. Instead of making the notebook more complicated with repeated steps, I learned the importance of keeping the code organized and checking the results after each operation. This helped me understand the data cleaning process better instead of just running code without knowing what each step was doing.

Another thing I learned was how useful charts can be when working with data. I created different visualizations to make the dataset easier to understand and to see how the values were distributed. Looking at charts helped me notice patterns that were not as obvious when I only looked at tables. I also learned that printing sample rows and checking the output after preprocessing are important because they help me see whether the data looks correct. Overall, this lab gave me more practice with Python, data cleaning, and visualization. I now understand that preparing data is not just about fixing missing values or running a few functions. It also means understanding the data, making reasonable decisions, and organizing the code so the notebook can run properly. These are skills I can use in future machine learning projects, especially when I start training models and working with larger datasets.

Montserrat's Reflection

During the Telco Customer Churn project, I worked on preparing and inspecting the dataset using Python in Google Collab. My tasks included loading the dataset, examining its structure, checking for missing and blank values, and preparing the data for the next stages of machine learning. I also added explanatory comments to notebook cells 1 through 6 to make the code easier to understand. Additionally, I worked with Pandas to create a report of missing

values and used Matplotlib to visualize null values and blank spaces by column. These tasks helped me understand the importance of checking data quality before building a machine learning model.

One challenge I encountered was understanding the purpose of each code section and how the steps connected to one another. I handled this challenge by reviewing the code, adding comments to explain the operations, and examining the outputs. Through this process, I learned the importance of preserving the original dataset, checking for missing values, and separating data into training and testing sets for supervised learning. I also learned that creating a missing-value report does not automatically correct the data; the appropriate preprocessing steps must be performed separately. Overall, this experience improved my understanding of Python, data visualization, and the importance of keeping a machine learning workflow organized and reproducible.

Improvements and Learning - Brian, Toan and Montserrat

In future projects, our group could improve the machine learning workflow by splitting the dataset before performing exploratory data analysis (EDA) to help prevent information leakage. We should use visualizations to compare data distributions before and after preprocessing, consider the information lost by deleting 1,000 rows, and evaluate whether our imputation and encoding choices introduce bias. If class balancing is necessary, we should apply techniques such as RandomOverSampler only to the training data to ensure a fair evaluation. We should also run the entire notebook from a fresh Google Collab runtime to confirm that every

step works correctly and the results are reproducible. These improvements will help me develop better data analysis skills, make more informed preprocessing decisions, and build more reliable and fair machine learning models in future projects.

Conclusion - Montserrat

Our Telco Customer Churn project helped us understand the importance of cleaning and preparing data before using it for machine learning. We used Python, Pandas, Matplotlib, and Seaborn to examine the dataset, identify missing values, correct data types, and visualize customer information. We also applied preprocessing techniques such as median and mode imputation, one-hot encoding, feature engineering, and numerical scaling. Additionally, we learned how to divide the data into training and testing sets and balance the training data to prepare it for a future machine learning model.

Throughout this project, we learned that every data-cleaning decision can affect the quality of the results. We also gained a better understanding of why removing CustomerID, handling missing values carefully, and preventing data leakage are important steps in building a reliable machine learning workflow. Although we faced some challenges with data preparation and organizing the notebook, working together helped us improve our Python skills, problem-solving abilities, and understanding of data quality. In future projects, we will focus on following the instructions carefully, checking our results after each stop, and ensuring that the notebook runs correctly from beginning to end.